Setup

Install it on the GPU server.

For: Engineers

One chapter of the handbook. The full handbook, with every chapter, is handbook.pdf in the same folder.

This guide installs secretary-keeper on one GPU server with Docker Compose. Follow the steps in order. Each step ends with a check; don't move on until it passes.

1. What you need

Item	Minimum	Notes
Granite server	An existing vLLM server running Granite 4.2 30B (about 58 GB in bf16, so one 80 GB GPU or two 48 GB)	Not part of this stack. See the next section
GPUs for this stack	1 smaller GPU for `embed` + `rerank`	Optional: 1 for `extract`, and 4 Blackwell or 8 Hopper for `escalate`
NVIDIA driver + Container Toolkit	Recent enough for your vLLM image	`docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi` must list your GPUs
Docker Engine + Compose v2	Compose 2.20 or later	`docker compose version`
Disk	About 400 GB	Mostly model weights
A machine with internet access	Temporary	To download images and weights before moving them in

Granite server requirements

The main model, IBM Granite 4.2 30B (Apache 2.0, dense, about 29B parameters, 128K context), is **not** started by this stack. It already runs on your own vLLM server. This stack calls it. Ask whoever runs that server to confirm each line:

Setting on the Granite server	Value	Why
vLLM version	0.20 or later	Earlier versions do not support Granite 4.2
Model name	`--served-model-name granite-4.2-30b`	Or keep your own name and set `GRANITE_MODEL` in `.env` to match
Precision	`--dtype bfloat16`	About 58 GB of weights

Setting on the Granite server	Value	Why
Tool calling	`--enable-auto-tool-choice --tool-call-parser qwen3_coder`	Hermes needs the model to call the policy tools
Reasoning	`--reasoning-parser nemotron_v3`	Built into vLLM. The model card's `granite_thinking_parser.py` plugin also works, but it runs code from the model download, so it is optional
Context length	`--max-model-len` of at least **65536**	Hermes refuses models under 64K. 131072 is recommended and matches `context_length` in `hermes/config.yaml`
Structured output	JSON-schema responses must work (vLLM guided decoding)	Every verdict is requested with `response_format` `json_schema`
API key	`--api-key <key>` (recommended)	Put the same key in `GRANITE_API_KEY`
Address	An `https://` address, unless the server runs on this same machine	Put it in `GRANITE_URL`, without `/v1`

Quick check, from a machine that can reach the server:

```
curl -s -H "Authorization: Bearer $GRANITE_API_KEY" $GRANITE_URL/v1/models
```

Check: the answer lists `granite-4.2-30b` (or the name you set in `GRANITE_MODEL`). Leave out the `-H` part if the server has no key.

The gateway

Nothing in the stack talks to the Granite server directly. The `granite-gateway` container (nginx, `gateway/granite.conf.template`) is the only container on the `llm-egress` network, and the only way out of the sealed backend network. It:

- passes on only `/v1/chat/completions` and `/v1/models`, and returns 404 for everything else;
- adds `GRANITE_API_KEY` itself and drops any key the caller sent, so the key lives in one place;
- checks the server's certificate. For a certificate from a private CA, put the CA file in `gateway/` (for example `gateway/granite-ca.pem`) and set `GRANITE_CA_PATH=/etc/nginx/granite/granite-ca.pem` in `.env`. `*.pem` files in `gateway/` are not committed to git.

`JUDGE_THINKING=1` makes Granite reason before each verdict. It is slower and sometimes better. The default is off. Try it on your evaluation cases first.

2. Bring images and weights into the environment

On a machine **with** internet access:

```
# Images: pin the exact tags you will write into .env
docker pull vllm/vllm-openai:<VLLM_TAG>
docker pull ghcr.io/huggingface/text-embeddings-inference:<TEI_TAG>
```

```
docker pull ghcr.io/open-webui/open-webui:<OPENWEBUI_TAG>
docker pull nousresearch/hermes-agent:<HERMES_TAG>
docker pull pgvector/pgvector:pg17
docker pull nginx:1.27-alpine
# Tag must equal APP_VERSION in .env, so Compose uses this image instead of trying to build one.
docker build -t secretary-keeper/policy-api:0.1.0 policy_api    # needs PyPI or your internal
mirror
docker save -o secretary-keeper-images.tar vllm/vllm-openai:<VLLM_TAG> ... secretary-
keeper/policy-api:0.1.0

# Weights (see models/README.md for the list)
hf download ibm-granite/granite-embedding-english-r2 --local-dir models/embedding
hf download ibm-granite/granite-embedding-reranker-english-r2 --local-dir models/reranker
# Granite itself stays on your vLLM server, so it is not copied here.
scripts/verify_models.sh record
```

Move the tar file, the repo and the `models/` folder into the environment. Then:

```
docker load -i secretary-keeper-images.tar
scripts/verify_models.sh check
```

Check: `verify_models.sh check` prints "All model files match their recorded hashes."

3. Configure

```
cp .env.example .env
for k in POSTGRES_PASSWORD POLICY_API_KEY LINK_SIGNING_KEY LLM_API_KEY OPENWEBUI_SECRET_KEY
HERMES_API_KEY; do
  sed -i "s/^$k=.*/$k=$(openssl rand -hex 32)/" .env
done
chmod 600 .env
```

Then edit `.env`:

- `GRANITE_URL`, `GRANITE_API_KEY`, `GRANITE_MODEL`: your Granite server (see above). Add `GRANITE_CA_PATH` if its certificate comes from a private CA.
- `VLLM_TAG`, `TEI_TAG`, `OPENWEBUI_TAG`, `HERMES_TAG`: the exact tags you loaded. `HERMES_TAG` was tested with `v0.22.3`.
- `HERMES_API_KEY`: at least 16 characters (the command above makes 64). Hermes will not start without it, and Open WebUI uses the same value to talk to Hermes.
- `PUBLIC_BASE_URL`: the address people's browsers use for downloads, e.g. `https://policy.plant.local:8100`.
- `POLICY_ID_PATTERN`: if your IDs don't look like `ABC-123`.
- `POLICY_SOURCE_DIR`: the folder holding your policy files.

Assign GPUs to the models this stack runs itself (embedding, reranker and any optional ones). Granite is not on this list. `compose.yaml` names no GPUs, so you choose them for this server:

```
python3 scripts/setup_gpus.py
```

It lists your GPUs, asks which ones each model uses, checks the plan (for example that no two big models share a GPU), writes `compose.gpu.yaml` and sets `COMPOSE_FILE`, `ESCALATE_TP` and the other size settings in `.env`. Run it again whenever the hardware changes. To do it by hand instead, follow the comments in `compose.gpu.example.yaml`, which also has sizing notes per model.

If your embedding model's output size is not 768, change `vector(768)` in `db/init.sql` now, before the database is created.

Check: `docker compose config --quiet` prints nothing.

4. Start the stack

```
docker compose up -d db embed rerank granite-gateway
docker compose logs -f embed rerank   # wait until both say they are ready
docker compose up -d policy-api hermes open-webui proxy
```

Optional models:

```
docker compose --profile escalate up -d escalate   # then set
ESCALATE_URL=http://escalate:8000/v1 in .env
docker compose --profile extract up -d extract     # then set EXTRACT_URL=http://extract:8000/v1
in .env
docker compose up -d policy-api                     # restart to pick up .env changes
```

Check: `docker compose ps` shows every service as running or healthy. To prove the gateway reaches Granite: `docker compose exec policy-api python3 -c "import urllib.request as u; print(u.urlopen('http://granite-gateway:8000/v1/models').read()[:200])"` prints the model list.

5. Prove the environment is contained

```
scripts/check_containment.sh
```

Check: it ends with `PASS contained`. Each service it checks must first reach an internal service (proving the probe works) and then fail to reach the internet. It also checks that `granite-gateway` is the only container on `llm-egress`. If it says FAIL, stop and fix it before loading real policies. See security.md.

6. Load policies

Policy files can be `.md`, `.txt`, `.docx` or `.pdf`. A short header block at the top of each file gives the details the system can't guess:

```
---
id: SAF-112
title: Maintenance Access
owner: Maintenance Manager
parent: POL-007
subsets: [Safety]
version: 1
---
```

Without a header, the ID is taken from the file name or the first lines, and the version counts up. Groups (`subsets`) must be in the header.

```
docker compose run --rm policy-api python -m app.ingest /policies
# Also use the extract model to find references the regex misses (slower):
docker compose run --rm policy-api python -m app.ingest /policies --llm-refs
```

Check: it prints `ingested <ID> v<N>` for each file and lists any skipped files with the reason.

7. Set up Hermes and Open WebUI

Hermes

Hermes needs no setup in its own screen. Its settings are files in `hermes/`, mounted read-only: `config.yaml` (model, allowed tools, limits), `SOUL.md` (its rules) and `plugins/secretary/` (the policy tools). Open WebUI is already pointed at it by `compose.yaml`: `OPENAI_API_BASE_URL` is `http://hermes:8642/v1`, `OPENAI_API_KEY` is `HERMES_API_KEY`, and the default model is `policy-assistant`.

If `GRANITE_MODEL` in `.env` is not `granite-4.2-30b`, change `model.default` and `delegation.model` in `hermes/config.yaml` to match, then `docker compose up -d hermes`.

Check that Hermes is up and sees only the policy tools:

```
docker compose logs hermes | tail -30     # look for the API server listening on port 8642 and
the secretary plugin loaded
docker compose exec hermes python3 -c "import urllib.request as u;
r=u.Request('http://localhost:8642/v1/models', headers={'Authorization':'Bearer
'+__import__('os').environ['API_SERVER_KEY']}); print(u.urlopen(r).read()[:300])"
```

The second command must list `policy-assistant`. Then, in a chat, ask *"What tools do you have?"*. The answer should name only policy tools: search, read, list groups, find, check, get result, make files, compare, make document, plus the to-do list and subagents. If it mentions web search, files, a terminal or memory, stop and check `agent.disabled_toolsets` and `platform_toolsets` in `hermes/config.yaml` (the file is read when Hermes starts).

If Hermes exits with a message about context size, the Granite server's `--max-model-len` is below 65536 (Granite server requirements).

Open WebUI

Open `http://<server>:3000`. The first account you create becomes the admin. Then:

1. **Turn off sign-up.** Set `OPENWEBUI_ENABLE_SIGNUP=false` in `.env` and run `docker compose up -d open-webui`. Add users from the Admin Panel.
2. **Add the upload filter** (ui.md, step 2) and set its `API_KEY` to `POLICY_API_KEY`.
3. **Pick the model.** People see one model, `policy-assistant`, which is Hermes. There is nothing else to install: the Open WebUI tool (`openwebui/policy_impact_tool.py`), the custom system prompt and "Function Calling: Native" are for the other versions and are **not used** here. Hermes calls the policy tools itself.

Menu names move between Open WebUI releases. If a step doesn't match, look for the same setting nearby. If uploads stop working after an Open WebUI upgrade, check the filter first (operations.md).

Check: in a new chat, ask *"Which Safety policies are affected by the change to POL-007?"* and attach the new version. You get an answer with a download link, and the link opens an Excel file.

8. Before real use

- [ ] `scripts/check_containment.sh` passes
- [ ] Evaluation on at least 10 past changes meets your recall target (evaluation.md)
- [ ] Security checklist done (security.md)
- [ ] Two or three staff have tried it using only how-to.md, without help